Swaps

This guide walks you through testing Ark/Lightning **submarine** and **reverse submarine** swaps against a full local regtest stack.

The stack is orchestrated end-to-end by the in-house `arkade-regtest` Node CLI (`regtest/regtest.mjs`, vendored as the `regtest` git submodule) — there is no Nigiri and no manual service wiring. A single `pnpm run regtest:start` brings up **and provisions** everything:

- **Ark** — Bitcoin Core + `arkd`/`arkd-wallet` (the server wallet is created, unlocked and funded automatically).
- **Boltz** — the Boltz backend, its Lightning node (`boltz-lnd`) and Fulmine (`boltz-fulmine`). The `boltz-lnd` ↔ `lnd` channel is opened and balanced, Boltz's Bitcoin Core wallet is funded, and the ARK/BTC pairs are verified.
- **Counterparty Lightning** — a second LND node (`lnd`) that you drive with `lncli` to play the remote side of each swap.
- **Arkade wallet** — the app under test (run with `pnpm start`).

  Everything the earlier (Nigiri-based) edition of this guide did by hand — `nigiri start --ln`, manual `arkd wallet` create/unlock, opening LND channels, wiring Fulmine — is now automatic. Funding is done with the CLI faucet: `node regtest/regtest.mjs faucet <address> <btc> --confirm`.

Requirements

- Docker
- Node.js v20.19+ or v22.12+ (drives the `arkade-regtest` stack via `regtest/regtest.mjs`)
- jq (optional — used below to pull fields out of `lncli` JSON)

1. Start the regtest stack

From the repo root:

```
pnpm run regtest:start
```

This runs `node regtest/regtest.mjs start --env .env.regtest` (the full stack, including the `boltz` profile) and the local nostr relay. The first run pulls images and can take a few minutes; it's ready when the CLI prints the service URLs (arkd on `http://localhost:7070`, Fulmine on `http://localhost:7003`, the Boltz CORS proxy on `http://localhost:9069`, …).

A handy alias for the counterparty Lightning node:

```
alias lncli="docker exec -i lnd lncli --network=regtest"
```

2. Start the wallet

```
pnpm start
```

Open http://localhost:3002 and create/unlock a wallet. On `localhost` the app automatically targets the local `arkd` (`http://localhost:7070`) and its regtest network — no `VITE_ARK_SERVER` needed. The regtest Boltz endpoint (`http://localhost:9069`) is likewise built in.

> To exercise the LNURL / nostr-backup features as well, start with: `VITE_LNURL_SERVER_URL=http://localhost:9090 VITE_NOSTR_RELAY_URL=ws://localhost:10547 pnpm start`

3. Fund the wallet

On the wallet's **Receive** screen, copy the on-chain (boarding) address — the `bcrt1…` one — and faucet it:

```
# 0.001 BTC; --confirm mines a block so the deposit confirms
node regtest/regtest.mjs faucet <address> 0.001 --confirm
```

Back on the wallet home, open the pending transaction and **settle** it. You're now ready to test swaps.

Test Submarine Swap (Ark → Lightning)

Create a 5,000-sat invoice on the counterparty node and pay it from the wallet:

```
lncli addinvoice --amt 5000 | jq -r .payment_request
```

Copy the `payment_request` and pay it in Arkade. After payment your transaction history shows the outgoing swap (amount + Boltz fee), and Fulmine's history (http://localhost:7003) shows the matching incoming movement.

> The exact sat split (amount vs. fee) depends on the Boltz release and on-chain fee rate, so don't expect fixed numbers — just that `amount + fees` reconciles.

Test Reverse Swap (Lightning → Ark)

In Arkade go to **Receive**, enter an amount (e.g. 4,000 sats) and continue to get a Lightning invoice. Pay it from the counterparty node:

```
lncli payinvoice --force <invoice>
```

The wallet should show the received funds a moment later.

Teardown

```
pnpm run regtest:stop      # stop the stack
pnpm run regtest:clean     # stop and wipe all volumes/state
```

Troubleshooting

- On Apple Silicon (M-family) you may need to build the boltz-backend image locally and point the stack at it:

  ```
  # Clone boltz-backend locally
  git clone git@github.com:BoltzExchange/boltz-backend.git && cd boltz-backend
  # Build the image; VERSION=ark builds the ark branch.
  docker build --build-arg NODE_VERSION=lts-bookworm-slim --build-arg VERSION=ark \
    -t boltz/boltz:ark -f docker/boltz/Dockerfile .
  ```

  Then set `BOLTZ_IMAGE=boltz/boltz:ark` in `.env.regtest` and re-run `pnpm run regtest:start`.